Introduction à la Programmation et l'Algorithmique

DINO LOPEZ PACHECO

DINO.LOPEZ@UNIV-COTEDAZUR.FR

Structures de données

UN PEU DE THÉORIE ☺

Tableaux

- Taille fixe
  - 0..n
- Insertion, lecture, modification par un index
  - Insertion O(1)
  - Recherche O(n)

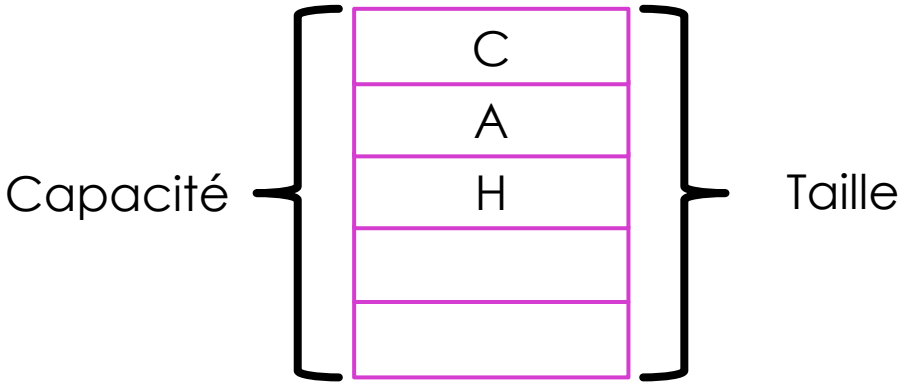
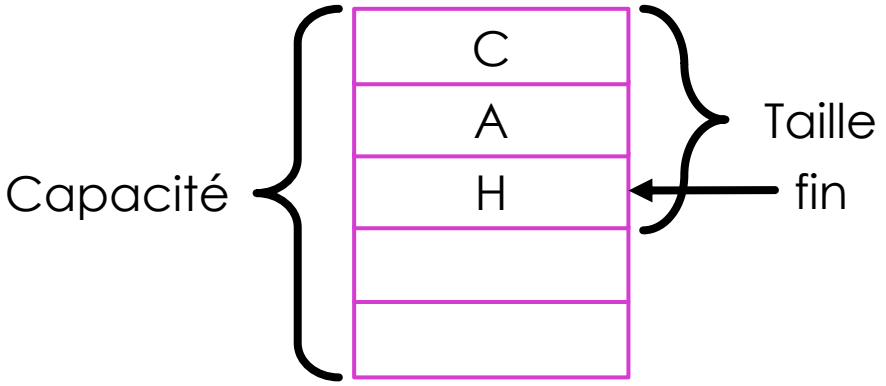


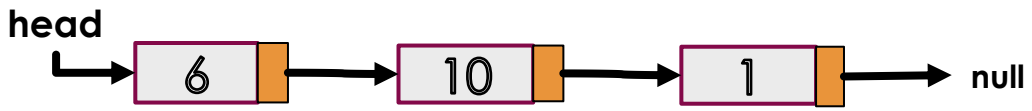
Tableau dynamique

- Il s'agit en réalité d'un tableau « normal », avec capacité fixe
  - Pointeur pour trouver la « fin » du tableau
- Si la capacité du tableau est atteint, une nouvelle insertion déclenche la création d'un tableau avec une capacité plus forte
  - Les données sont copiées du tableau petit au tableau grand
  - O(n)

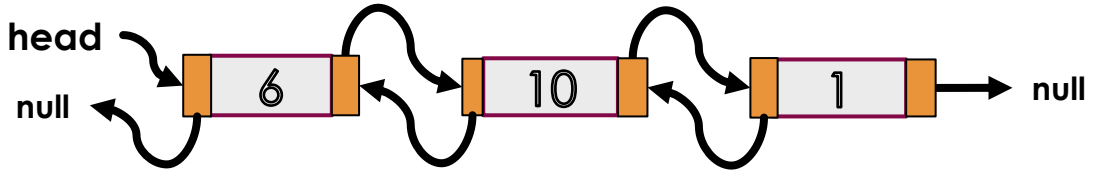


Listes chaînées

- Listes simplement chaînées



- Listes doublement chaînes



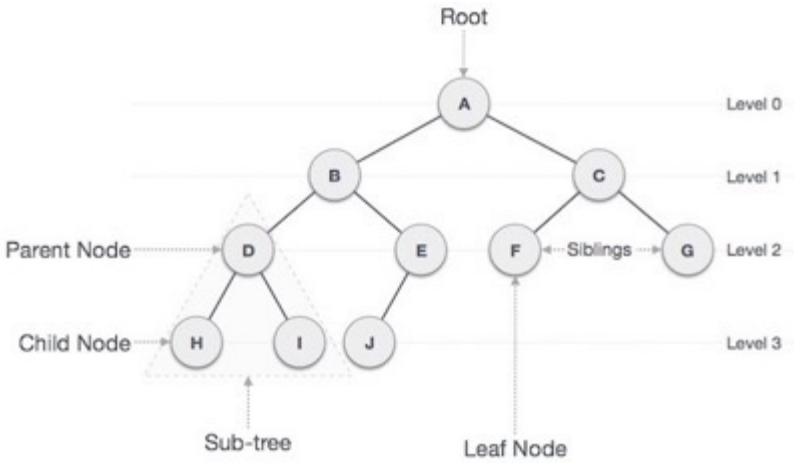
  - Insertion et suppression d'éléments facile
    - Insertion en début de liste O(1)
    - Insertion en fin de liste O(n)
  - Recherche lente d'éléments -> O(n)
    - Dans une liste simplement chaînées : recherche en seul sens
    - Dans une liste simplement chaînées : recherche dans les 2 sens

Hash

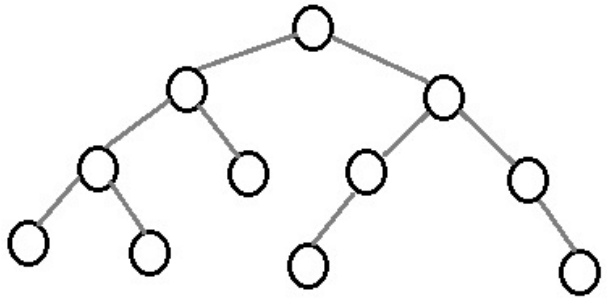
- Une fonction d'hachage est utilisée pour calculer une clé (index où garder la donnée).
  - Insertion, effacement et recherche rapide
    - De manière optimale : O(1)
- Possibilité de collisions :
  - 2 données distinctes peuvent donner lieu à une même clé
  - Liste chaînée dans une table de type *hash*
  - Moins approprié pour des large collection de données

01	
02	
03	10
04	
05	
06	1
07	
08	5 -> 33 ->44
09	
0A	

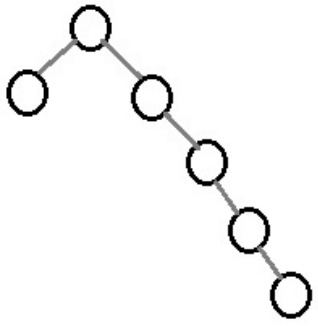
Arbres



- Addition, effacement et recherche de données moins efficace que dans une table *hash*, mais plus rapide que dans une liste
  - Elimination de branches (pruning)
- Très utilisés dans le domaine du *Machine Learning*
- Enormément de choses à dire et apprendre sur les arbres… mais pas ici ;)

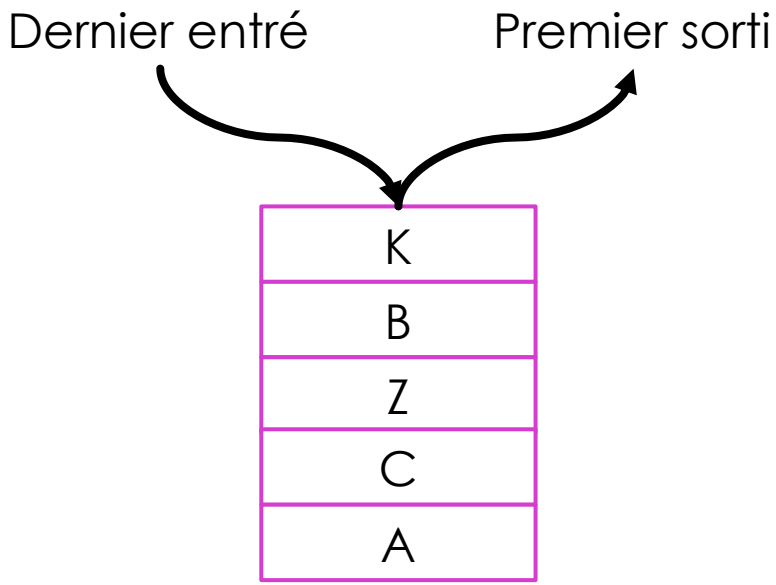


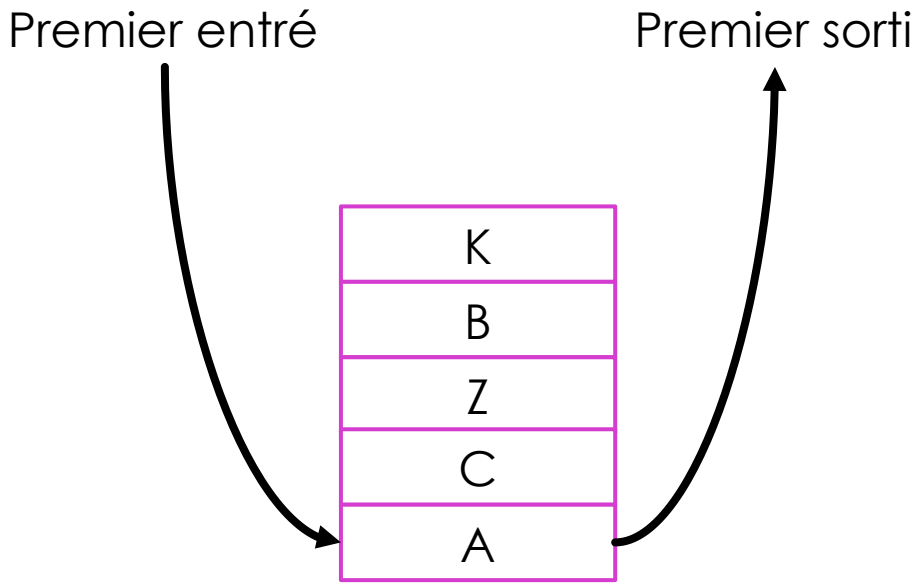
Balanced Binary Tree



Unbalanced Binary Tree

Piles & Queues





Tableaux associatifs

- Pas vraiment une structure de donnée, mais un type de donnée
  - Sa réalisation a besoin d'un support matériel
  - Peut être émulé par des structures *hash* et des arbres
- La clé n'est pas un index, mais un type arbitraire de donnée
  - La clé et la valeur associé peuvent être de type différents
- Aussi appelé table d'association, dictionnaire, ou encore un *map*

"bleu"	34523
"orange"	87645
"caramel"	56783
"melon"	34512

Les Tableaux

Les tableaux en Java

- Un tableau est un objet qui contient un nombre fixe d'éléments d'un même type
  - La création de l'objet utilise le mot clé « new »
  - La taille du tableau est fixé lors de sa création
- Les tableaux peuvent avoir des multiples dimensions
- Pour accéder aux éléments d'un tableau, on utilise des index
  - Le première élément d'un tableau se trouve à l'index 0 et le dernier à la position « n-1 »

Les tableaux en Java

- Exemples de déclaration d'un tableau
  - ```
    int[] tableau1;
    tableau1 = new int[10];
    ```
  - ```
    String[] noms = {"nom1","nom2","nom3"};
    ```
  - ```
    int[][] test = {{0,1},{10,11,12,13},{20,21,22}};
    ```
  - ```
    Point[][] pts = new Point[10][];
    ```
- Exemples d'accès aux tableaux
  - ```
    tableau1[0]=5;
    ```
  - ```
    noms[2] = "nomx";
    ```
  - ```
    System.out.println(test[1][3]); //13
    ```
    ```

Parcourir des tableaux

Style for-each

```
for (int val : tableau1) {
    System.out.println(val);
}

for (String nom : noms) {
    //useful code
}

for (??? x : test) {
    …
}
```

Avec l'attribut « length »

```
for (int i = 0; i < tableau1.length;
i++) {
    System.out.println(tableau[i]);
}

??? x;
for (int i = 0; i < test.length; i++)
{
   x = test[i];
}
```

Manipulation de tableaux*

- Pour copier 2 tableaux efficacement
  - System.arraycopy()
  - copyOfRange() de la classe « java.util.Arrays »
- Autres méthodes depuis « java.util.Arrays »
  - Chercher une valeur dans un tableau et obtenir son index : binarySearch()
  - Comparer 2 tableaux : equals()
  - Remplir un tableau avec une valeur quelconque : fill()
  - Trier un tableau en ordre ascendent : sort() -séquentiel-, parallelSort() –trie parallèle-

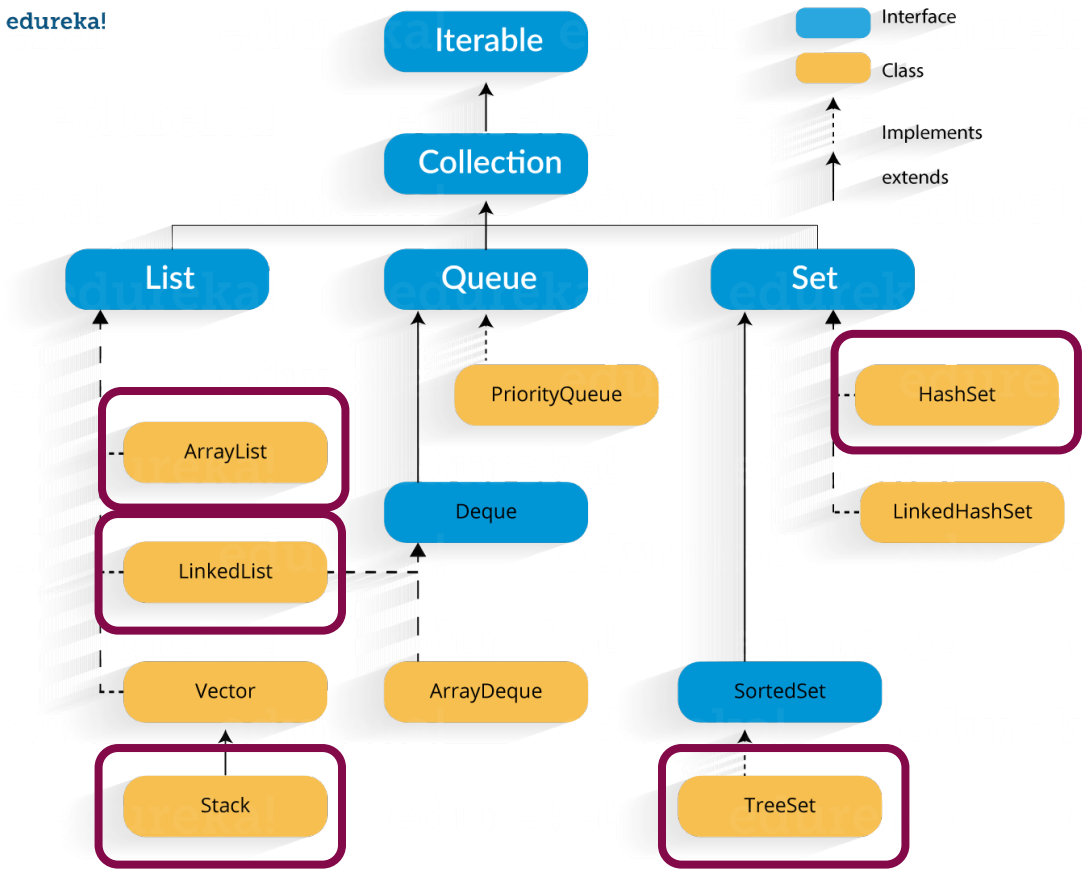
* https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Les collections de Données en Java

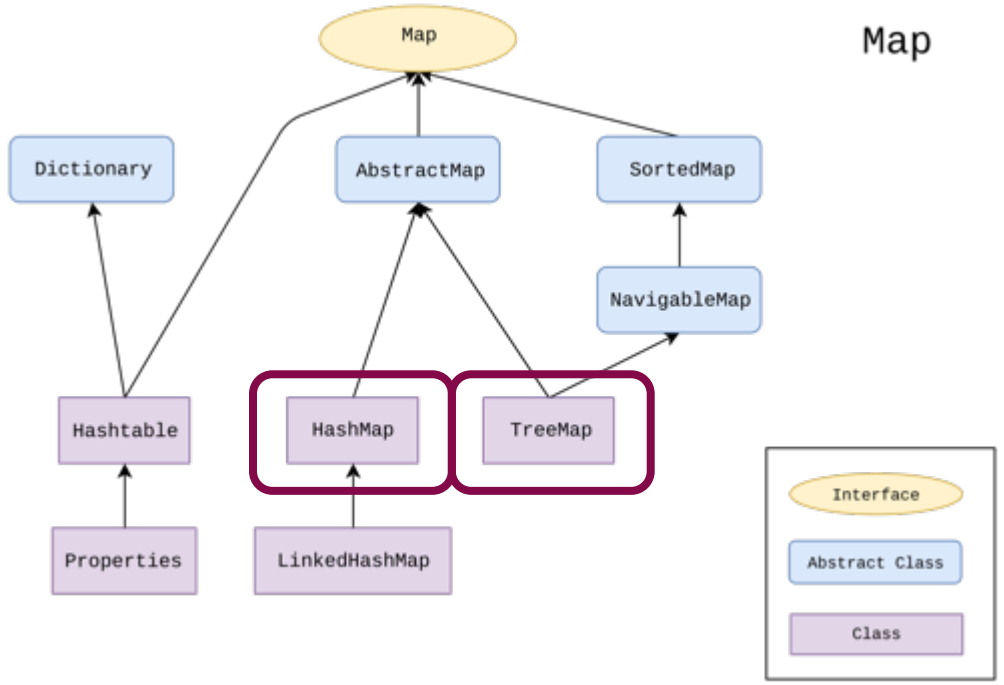
Qu'est-ce une collection de données en Java

- Framework qui définit une architecture pour créer et manipuler un groupe d'objets
  - Insertion, effacement, modification
  - Recherche, trie.
  - Implémentation des structures de données
- Les classes qui implémentent les collections de données
  - contiennent uniquement des objets
  - Impossibilité de contenir de types de données primitifs
  - Les types primitifs doivent être enveloppés dans leur objets équivalents
    - E.g. int -> Integer, double -> Double, etc. etc.
- Possibilité de parcourir les collections de données avec des itérateurs (plus d'info plus tard)

List, Queue, Set & Map



From https://www.edureka.co/blog/java-collections/



By ramlmn - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=64044187

Les tableaux dynamiques - ArrayList

« java.util.ArrayList »

Opération basiques d'ArrayList

- Déclaration et initialisation
  - ArrayList<TypeObjet> al;
    al = new ArrayList<TypeObjet> ();
  - ArrayList<String> al;
    al = new ArrayList<String> ();
- Ajouter un élément
  - al.add("test");
- Lire un élément
  - al.get(0);
- Modifier un élément
  - al.set(0,"aux");
- Effacer un élément
  - al.remove(0);
- effacer tous les éléments
  - al.clean();
- Taille d'un ArraList
  - al.size();

Parcourir un ArrayList

Style for-each

- Similaire à ce qu'on fait pour les tableaux

```
for (String elem : al) {
    System.out.println(elem);
}
```

Avec un for (ou while, au choix)

```
for (int i = 0; i < al.size(); i++) {
    System.out.println(al.get(i));
}
```

Les collections de données gardent uniquement des objets

- Le code à droite fonctionne
  - Notez bien la déclaration d' « Integer », pas « int »
- Encapsulation/décapsulation automatique
  - Encapsulation (int -> Integer)
  - Décapsulation (Integer -> int)
- Solution simple, mais plus couteux que si on utilisait des tableaux pour les types primitifs

```
ArrayList<Integer> myNumbers = new
ArrayList<Integer>();
myNumbers.add(10);
myNumbers.add(15);
myNumbers.add(20);
myNumbers.add(25);
for (int i : myNumbers) {
    System.out.println(i);
}
```

HashSet

java.util.HashSet

Les collections d'objets Set

- Set
  - Collections d'objets non ordonnées
  - Objets non dupliqués
    - Naturellement adapté aux structures de données hash -> HashSet
    - Implémentation en arbre également existante -> TreeSet
- HashSet -> ordre d'insertion non-maintenu
  - Normal, c'est la fonction *hash* qui indique où garder un élément
- LinkedHashSet -> ordre d'insertion maintenu grâce à une liste doublement chaînée

Manipulation d'un objet HashSet

- Déclaration
  - HashSet() / HashSet(int initialCapacity)
  - HashSet<Objet> hs = new HashSet<Objet>(100); // tableau hash avec capacité pour 100 éléments
  - HashSet<String> hss = new HashSet<String>();
- Ajouter et effacer un élément
  - hss.add("exemple");
  - hss.remove("exemple");
- Rechercher un élément
  - hss.contains("exemple");
  - contains() renvoie un type boolean
- Nombre d'éléments dans un HashSet
  - hss.size();
- Conversion d'un hashset en tableau fixe avec la méthode « toArray() »
  - toArray() renvoie un tableau Object
  - Object[] objTab = hss.toArray();
- Parcourir un HashSet
  - Style for-each
  - Autres manière possibles (e.g. après transformation en tableau)

HashSet - Exemple

```
HashSet<Integer> hsi = new HashSet<Integer>();
hsi.add(10);
hsi.add(200);
hsi.add(3000);
hsi.remove(3000);
System.out.println(hsi.contains(3000)); // affiche « false »
System.out.println(hsi.contains(200)); // affiche « true »
System.out.println(hsi.size()); // affiche 2
for (int val : hsi)
    System.out.println(val);  // affiche chaque élément
```

HashMap

java.util.HashMap

java.util.Map

La collection d'objets Map

- La collection Map permets de « mapper » une clé (K) quelconque à une certaine valeur (V)
  - La clé ne doit pas se répéter
  - Tableau associatif
- Pareil que Set, Map est un bon candidat pour une implémentation avec des structure hash
  - Mais sa variante en arbre (Tree) existe également

Manipulation d'un HashMap

- Déclaration
  - HashMap() / HashMap(int initialCapacity)
  - HashMap<ObjetK, ObjetV> hm = new HashMap<ObjetK, ObjectV>();
  - HashMap<String, Integer> hm = new HashMap<String, Integer>();
- Ajouter et mettre à jour un élément : put()
  - hm.put("cle1",123); hm.put("cle2",231);
- Effacer un élément : remove(K), remove(K,V)
  - hm.remove("cle1"); hm.remove("cle2",231);
- Nombre de couples « K,V »
  - size()
- Parcourir un HashMap
  - Besoin de le transformer en une collection Set avec « entrySet() »
  - entrySet() envoie un ensemble d'objets Map.Entry<K,V>
- Retrouver une valeur associée à une clé
  - get()

HashMap - Exemple

```
HashMap<String,Integer> hm = new HashMap<String,Integer>();
hm.put("Paris",24);
m.put("Madrid",32);
hm.put("Oslo",10);
hm.put("Oslo",11); // met à jour ("Oslo",10) -> ("Oslo",11)
hm.put("Stockholm",10);
System.out.println(hm.size()); // affiche 4
Set<Map.Entry<String,Integer>> s = hm.entrySet();
// boucle sur le Set, comme fait avec HashSet
for (Map.Entry<String,Integer> me : s)
    System.out.println("K =  " + me.getKey() +  ", V = " + me.getValue());
hm.remove("Stockholm");
hm.remove("Oslo",11);
```

Iterator

java.util.Iterator

Parcourir les collections de données

- Les collections de données Java implémentent la classe « Iterator »
  - Parcourir les collections de données sous un seul framework
  - Méthode « iterator() »
- Utilisation des boucles for-each
  - Utilisation d' « Iterator » en *backstage*
  - *Syntactic sugar*
- Utilisation directe d' Iterator
  - Possibilité de modifier la collection de données sous-jacente
- Attention : Map n'appartient pas aux collection de données Java !
  - Rappel : on obtient un Set pour parcourir les données de type Map

Exemple d'Iterator

- itérateur -> liste chaînées
  - Un objet pointe vers le prochain élément
    - hasNext() -> fin des données ?
    - next() -> obtien le prochain élément
  - Dernier élément pointe à « null »
- D'ailleurs, « ListIterator » existe -> liste doublement chaînée
  - Parcourir la collection dans les 2 sens

```
// hm -> (Oslo 11),(Madrid 32),(Paris 24)
Iterator it = hm.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry me = (Map.Entry)it.next();
    if (me.getKey().equals("Oslo"))
        it.remove();
    System.out.println(me.getKey());
}
//hm -> (Madrid 32),(Paris 24)
```